



دانشکده مهندسی کامپیوتر

پروژه درس بینایی کامپیوتر

عارفه آتشگر

پری قاسمی

فاز اول پروژه :

فاز ۱ – استخراج جدول سودوکو

- تبدیل تصویر به Grayscale
- حذف نویز و استخراج لبه‌ها
- تشخیص جدول با:
 - findContours
 - Hough Transform یا
- اعمال Perspective Transform
- تقسیم جدول به ۸۱ سلول
- حذف خطوط جدول و تشخیص خانه‌های خالی
- تحویل:
 - تصاویر مراحل پردازش
 - نمونه خروجی سلول‌ها
 - تحلیل نمونه‌های شکست

```
run_id = datetime.now().strftime("%Y%m%d_%H%M%S")
debug_dir = Path(args["debug_root"]) / run_id
debug_dir.mkdir(parents=True, exist_ok=True)

img = cv2.imread(img_fpath)
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
img = sutils.resize_and_maintain_aspect_ratio(img, new_width=1000)
```

توی این قسمت در فایل sudoku_main.py برای هر اجرا یک پوشه مستقل در debugs_run ساخته می‌شه. تصویر رو می‌خونیم. ترتیب کانال‌ها رو به rgb تغییر می‌دیم. عرض تصویر به ۱۰۰۰ پیکسل می‌رسه و نسبت طول به عرض حفظ می‌شه.

```

try:
    cells, M, board_image, diagnostics = sutils.get_valid_cells_from_image(
        img,
        debug_dir=debug_dir,
        return_diagnostics=True,
    )

```

در اینجا این تابع فراخوانی می‌شود که تقریباً همه کارهای فاز ۱ توی اون انجام می‌شه.

```

def resize_and_maintain_aspect_ratio(input_image, new_width):
    """
    A simple utility function to resize an image by specifying a new width,
    while maintaining the original aspect ratio.

    Args:
        input_image: Array containing image data.
        new_width: Desired width of the resulting reshaped image.

    Returns:
        reshaped_image: Array containing reshaped image data.

    """

    # Resize maintaining aspect ratio
    orig_width, orig_height = input_image.shape[1], input_image.shape[0]
    # Determine the aspect ratio
    ratio = new_width / float(orig_width)
    # Calculate the new height to match new_width, given aspect ratio
    new_height = int(orig_height * ratio)
    dim = (new_width, new_height)
    reshaped_image = cv2.resize(input_image, dim, interpolation=cv2.INTER_AREA)

    return reshaped_image

```

در ابتدای پردازش، تصویر ورودی با حفظ نسبت طول به عرض به عرض ثابت ۱۰۰۰ پیکسل تبدیل شد. استانداردسازی ابعاد تصویر موجب می‌شود پارامترهای پردازش تصویر، از جمله آستانه مساحت کانتورها، در تصاویر مختلف رفتار یکنواخت‌تری داشته باشند.

```
def apply_grayscale_blur_and_threshold(img, method="mean", blockSize=91, c=7):
    img = cv2.GaussianBlur(img, ksize=(3, 3), sigmaX=0)
    gray = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)

    if method == "mean":
        adaptiveMethod = cv2.ADAPTIVE_THRESH_MEAN_C
    elif method == "gaussian":
        adaptiveMethod = cv2.ADAPTIVE_THRESH_GAUSSIAN_C
    thresh = cv2.adaptiveThreshold(gray,
                                   maxValue=255,
                                   adaptiveMethod=adaptiveMethod,
                                   thresholdType=cv2.THRESH_BINARY,
                                   blockSize=blockSize,
                                   C=c)

    thresh = cv2.bitwise_not(thresh)

    return thresh
```

به منظور کاهش اثر نویزهای تصویری، ابتدا فیلتر Gaussian با اندازه کرنل 3×3 روی تصویر اعمال شد. سپس تصویر از فضای رنگی RGB به تصویر خاکستری تبدیل کردید. برای جداسازی خطوط و ارقام از پس زمینه، از آستانه گذاری تطبیقی استفاده شد. برخلاف آستانهگذاری سراسری، این روش مقدار آستانه را براساس همسایگی هر پیکسل تعیین می کند و در برابر تغییرات روشنایی تصویر مقاوم تر است. در پایان، تصویر باینری معکوس شد تا خطوط جدول و ارقام به رنگ سفید و پس زمینه به رنگ سیاه نمایش داده شوند.

```
# Apply blur, grayscale and adaptive threshold
thresh = apply_grayscale_blur_and_threshold(img, blockSize=41, c=8)

# Extract edges with Canny as an additional contour source.
# Adaptive-threshold contours are retained because they are often more
# reliable when the grid lines are already closed.
edges = cv2.Canny(thresh, threshold1=50, threshold2=150)
kernel = np.ones((3, 3), np.uint8)
edges = cv2.dilate(edges, kernel, iterations=1)
```

```
for source_img in (thresh, edges):
    detected = cv2.findContours(image=source_img.copy(),
                                mode=cv2.RETR_EXTERNAL,
                                method=cv2.CHAIN_APPROX_SIMPLE)
    detected = imutils.grab_contours(detected)
    contours.extend(detected)
```

پس از باینری سازی، لبه های تصویر با الگوریتم Canny استخراج شدند. دو مقدار آستانه ۵۰ و ۱۵۰ به ترتیب برای تعیین لبه های ضعیف و قوی در نظر گرفته شد. برای متصل کردن ناپیوستگی های کوچک در خطوط جدول، عملیات اتساع با کرنل 3×3 اعمال گردید. کانتورها از هر دو تصویر آستانه گذاری شده و خروجی Canny استخراج شدند تا مقاومت الگوریتم در برابر تفاوت کیفیت تصاویر افزایش یابد.

```
contours = sorted(contours, key=cv2.contourArea, reverse=True)
```

```
perimeter = cv2.arcLength(contour, True)
approx = cv2.approxPolyDP(contour, 0.03 * perimeter, True)
```

```
# We are looking for a quadrilateral contour with sufficiently large area
if len(approx) == 4 and contour_fractional_area > 0.1:
    # Get the corners in the required order
```

کانتورهای خارجی تصویر با تابع findcontours استخراج شدند. کانتورها براساس مساحت به صورت نزولی مرتب شدند و با استفاده از تابع approxPolyDP شکل تقریبی آن ها محاسبه شد. کانتوری به عنوان نامزد جدول پذیرفته شد. که چهار رأس داشته و مساحت آن بیش از ۱۰ درصد مساحت کل تصویر باشد. این شرط از انتخاب اشیای کوچک، متنهای جانبی و عناصر پس زمینه به عنوان جدول جلوگیری میکند.

```
def get_quadrilateral_points_in_order(approx_arr):

    try:
        assert(approx_arr.shape == (4, 1, 2) or approx_arr.shape == (4, 2))
    except:
        raise ValueError(f"Incorrect shape for approx_arr: {approx_arr.shape}. Requires shape of (4, 1, 2) or (4, 2).")

    if approx_arr.shape == (4, 1, 2):
        approx_arr = np.squeeze(approx_arr, axis=1)

    max_x = int(1.1 * np.max(approx_arr[:,0]))
    origin_1 = [0, 0]
    origin_2 = [max_x, 0]
    distances_1 = [np.linalg.norm(point - origin_1) for point in approx_arr]
    distances_2 = [np.linalg.norm(point - origin_2) for point in approx_arr]
    tl_idx = np.argmin(distances_1)
    br_idx = np.argmax(distances_1)

    dist_arr = distances_2.copy()
    dist_arr[tl_idx] = np.inf
    dist_arr[br_idx] = np.inf
    tr_idx = np.argmin(dist_arr)

    dist_arr = distances_2.copy()
    dist_arr[tl_idx] = -np.inf
    dist_arr[br_idx] = -np.inf
    bl_idx = np.argmax(dist_arr)
    tl = approx_arr[tl_idx]
    br = approx_arr[br_idx]
    tr = approx_arr[tr_idx]
    bl = approx_arr[bl_idx]

    return np.array([tl, tr, br, bl])
```

این تابع ۴ راس کانتور رو به ترتیب زیر مرتب می کنه:

بالا-چپ، بالا-راست، پایین-راست، پایین-چپ

فاصله هر راس نسبت به دو مبدا بررسی میشه، نزدیک ترین نقطه به مبدا اول: بالا-چپ، دورترین نقطه از مبدا اول: پایین-راست، دو نقطه دیگر: بالا-راست و پایین-چپ.


```

# of approximately 1% of the grid area (0.5% to 1.5% works well)
if len(approx) == 4 and contour_fractional_area > 0.005 and contour_fractional_area < 0.015:
    # We have found a valid contour
    # Use a mask to extract the identified cell from grid_img
    mask = np.zeros_like(grid_img)
    # Use the contour to mask out the cell
    cv2.drawContours(image=mask,
                     contours=[contour],
                     contourIdx=0,
                     color=255,
                     thickness=cv2.FILLED)

    # Get the indices where the mask is white
    y_px, x_px = np.where(mask==255)
    # Use these indices to crop out the cell from the image
    cell_image = grid_img[min(y_px):max(y_px)+1, min(x_px):max(x_px)+1]

```

```

# Resize the cell image to be 28x28 pixels for classification later
cell_image = cv2.resize(cell_image, dsize=(28, 28), interpolation=cv2.INTER_AREA)

```

برای استخراج سلولها، کانتورهای داخلی تصویر جدول اصلاح شده باحالت RETR_TREE شناسایی شدند. کانتورهای چهارضلعی که مساحت آن ها بین ۰٫۵ تا ۱٫۵ درصد مساحت کل جدول بود، به عنوان سلول معتبر انتخاب شدند. برای هر کانتور یک ماسک باینری ایجاد شد و ناحیه متناظر با سلول از تصویر جدول برش داده شد. در نهایت تصویر هر سلول به ابعاد 28×28 پیکسل تبدیل شد تا با ورودی شبکه عصبی سازگار باشد.

```

if apply_border:
    # Crude way to eliminate the unwanted pixels around the borders
    border_fraction = 0.07
    replacement_val = 0

    y_border_px = int(border_fraction * cell_img.shape[0])
    x_border_px = int(border_fraction * cell_img.shape[1])

    cell_img[:, 0:x_border_px] = replacement_val
    cell_img[:, -x_border_px:] = replacement_val
    cell_img[0:y_border_px, :] = replacement_val
    cell_img[-y_border_px:, :] = replacement_val

```



```

# Sort the contours according to contour area, largest first
contours = sorted(contours, key=cv2.contourArea, reverse=True)
largest_contour_area = cv2.contourArea(contours[0])
image_area = cell_img.shape[0] * cell_img.shape[1]
contour_percentage_area = 100 * largest_contour_area / image_area

if contour_percentage_area > area_threshold:
    image_contains_digit = True
else:
    image_contains_digit = False

else:
    image_contains_digit = False

return image_contains_digit, cell_img

```

در مرحله استخراج سلول، یک تشخیص مقدماتی مبتنی بر مساحت کانتور برای تفکیک خانه خالی از خانه دارای رقم انجام شد. در نسخه نهایی، تصمیم نهایی خالی یا پر بودن خانه توسط مدل CNN ده کلاس گرفته می شود.

```

M_matrices, warped_images, grid_candidates = find_grid_contour_candidates(img)

```

```

for i, grid_image in enumerate(warped_images):
    valid_cells = locate_cells_within_grid(grid_image)

```

```

if len(valid_cells) == 81:
    valid_cells = sort_cells_into_grid(valid_cells)

```

بنابراین یک نامزد فقط زمانی به عنوان جدول نهایی پذیرفته می شود که دقیقاً ۸۱ سلول داشته باشد.

```

# Reshape to give an array with 9 rows
rows = np.reshape(points_sorted, (9, 9, 2))
# Sort by y value for every row in rows
final = np.array([sorted(row, key=lambda x: x[0]) for row in rows])

```

برای هر کانتور نامزد جدول، سلول های داخلی استخراج شدند. تنها نامزدی معتبر در نظر گرفته شد که دقیقاً شامل ۸۱ سلول باشد. سپس مرکز هندسی هر سلول محاسبه و سلول ها ابتدا براساس مختصات عمودی در ۹ ردیف و سپس در هر ردیف براساس مختصات افقی مرتب شدند. در نتیجه، ترتیب نهایی سلول ها با ترتیب خواندن طبیعی جدول، از چپ به راست و از بالا به پایین، منطبق گردید.

```

gray = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
thresh = apply_grayscale_blur_and_threshold(img, blocksize=41, c=8)
edges = cv2.Canny(thresh, 50, 150)
edges = cv2.dilate(edges, np.ones((3, 3), np.uint8), iterations=1)

```

```

_save_rgb_image(debug_path / "01_input.png", img)
_save_rgb_image(debug_path / "02_grayscale.png", gray)
_save_rgb_image(debug_path / "03_threshold.png", thresh)
_save_rgb_image(debug_path / "04_canny_edges.png", edges)

```

```

if debug_path:
    contour_preview = img.copy()
    cv2.drawContours(contour_preview, grid_candidates, -1, (0, 255, 0), 3)
    _save_rgb_image(debug_path / "05_grid_candidates.png", contour_preview)

```

```

if debug_path:
    _save_rgb_image(debug_path / f"06_warped_candidate_{i + 1}.png", grid_image)
    save_cells_montage(valid_cells, debug_path / f"07_cells_candidate_{i + 1}.png")

```

```

if debug_path:
    save_cells_montage(valid_cells, debug_path / "08_cells_sorted_9x9.png")

```

این بخش دقیقاً قسمت تصاویر مراحل پردازش و نمونه خروجی سلول‌ها توی فاز اول رو پوشش می‌ده.

```
def analyze_image_quality(img):
    """Return simple, explainable image-quality measurements.

    The thresholds are practical heuristics and can be tuned for a particular
    camera or dataset.
    """
    gray = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
    blur_score = float(cv2.Laplacian(gray, cv2.CV_64F).var())
    brightness = float(np.mean(gray))
    contrast = float(np.std(gray))

    warnings = []
    if blur_score < 80:
        warnings.append("image appears blurry")
    if brightness < 45:
        warnings.append("image is too dark")
    elif brightness > 210:
        warnings.append("image is overexposed")
    if contrast < 25:
        warnings.append("image has low contrast")

    return {
        "blur_score": round(blur_score, 2),
        "brightness": round(brightness, 2),
        "contrast": round(contrast, 2),
        "quality_warnings": warnings,
    }
```

برای بررسی عملکرد سامانه در شرایط نامناسب، معیارهایی شامل میزان تاری، میانکین روشنایی، کنتراست و زاویه چرخش جدول محاسبه شدند. میزان تاری با واریانس لاپلاسین، روشنایی با میانکین شدت پیکسل ها و کنتراست با انحراف معیار شدت پیکسلها اندازهگیری شد. در صورتی که جدول شناسایی نشود یا تعداد سلول های استخراج شده کمتر یا بیشتر از ۱۸ باشد، وضعیت اجرا به صورت شکست ثبت شده و علت آن همراه با معیارهای کیفیت تصویر در فایل های JSON و CSV ذخیره می شود.

برای فاز دوم پروژه:

```
# Load the data and split it between train and test sets
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()
```

تابع mnist رو بارگذاری می کنیم.

```
# Remove the zero images - zero is an invalid sudoku entry
non_zero_train_indices = np.where(y_train != 0)[0]
non_zero_test_indices = np.where(y_test != 0)[0]

x_train, y_train = x_train[non_zero_train_indices], y_train[non_zero_train_indices]
x_test, y_test = x_test[non_zero_test_indices], y_test[non_zero_test_indices]

# Create a validation set
x_train, x_val, y_train, y_val = train_test_split(x_train,
                                                    y_train,
                                                    train_size=0.85,
                                                    random_state=2023)
```

نمونه‌های رقم ۰ حذف می‌شوند. علت حذف صفر این است که صفر نوشته شده یک رقم واقعی است و نباید با «خانه خالی» یکسان در نظر گرفته شود. کلاس صفر در مرحله بعد با تصاویر خانه‌های خالی ساخته می‌شود.

```
# Create a validation set
x_train, x_val, y_train, y_val = train_test_split(x_train,
                                                    y_train,
                                                    train_size=0.85,
                                                    random_state=2023)
```

بعد داده هامون رو تقسیم می‌کنیم به ۸۵ درصد داده آموزشی و ۱۵ درصد داده ولیدیشن.

```
# Scale images to the [0, 1] range
x_train = x_train.astype("float32") / 255
x_val = x_val.astype("float32") / 255
x_test = x_test.astype("float32") / 255
```

مقادیر پیکسل در تصویر معمولاً بین صفر تا ۲۵۵ هستند. پس از این عملیات، دامنه داده‌ها به بازه صفر تا یک تبدیل می‌شود.

```
def get_font_image_dict(excluded_names=None):
    """
    Load the font images and store in a dictionary whose keys are the digit
    and the values are the image pixel arrays converted to grayscale and resized
    to match the MNIST digit shape.

    """

    digit_image_filepaths = [
        glob.glob(f"data/digit_images/{digit}/*.png")
        for digit in range(1, 10)
    ]

    if excluded_names:
        # Exclude the unwanted files
        inclusion_list_indices = list(np.where([not any(elem in fpath for elem in excluded_names) for fpath in digit_image_filepaths]))
        digit_image_filepaths = [[fpath_list[i] for i in inclusion_list_indices] for fpath_list in digit_image_filepaths]

    # Create dictionary and load images from file
    img_dict = {i: None for i in range(1, 10)}
    for k in img_dict:
        img_dict[k] = [cv2.imread(fpath) for fpath in digit_image_filepaths[k-1]]

    # Convert all images to grayscale and resize to 28x28 px
    for k, v in img_dict.items():
        gray = [cv2.cvtColor(arr, cv2.COLOR_BGR2GRAY) for arr in v]
        img_dict[k] = [cv2.resize(arr, (28, 28), interpolation=cv2.INTER_AREA) for arr in gray]
        # Add "channels" dimension
        img_dict[k] = np.expand_dims(img_dict[k], -1)

    return img_dict
```

ساختار پوشه داده ها این طوری می‌شه که:

Data/digit_images/1/

2/

3/

...

بعد تصاویر خوانده می‌شن ، خاکستری می‌شن و به ۲۸ در ۲۸ تبدیل می‌شن.

```

def load_font_image_arrays(image_dict):
    # Create x array with all image data
    x = np.array([v for v in image_dict.values()])
    x = np.reshape(x, (-1, 28, 28, 1))
    # Create label array
    y = np.array([np.repeat(k, len(image_dict[k])) for k in image_dict])
    y = np.reshape(y, (-1, 1))
    # Convert y to one-hot labels. Exclude zeros - invalid sudoku entry
    y = keras.utils.to_categorical(y, num_classes=10)[: , 1:]

    # Split into train, validation and test sets with shuffle
    x_train, x_test, y_train, y_test = train_test_split(x,
                                                         y,
                                                         test_size=0.15,
                                                         shuffle=True,
                                                         random_state=0)

    x_train, x_val, y_train, y_val = train_test_split(x_train,
                                                         y_train,
                                                         test_size=0.18,
                                                         shuffle=True,
                                                         random_state=33)

    # Invert images so backgrounds are black and fonts are white
    x_train = np.array(list(map(cv2.bitwise_not, x_train)))
    x_val = np.array(list(map(cv2.bitwise_not, x_val)))
    x_test = np.array(list(map(cv2.bitwise_not, x_test)))

    # Scale images to [0, 1]
    x_train = x_train.astype("float32") / 255
    x_val = x_val.astype("float32") / 255
    x_test = x_test.astype("float32") / 255

    # Add the "channels" dimension
    x_train = np.expand_dims(x_train, -1)
    x_val = np.expand_dims(x_val, -1)
    x_test = np.expand_dims(x_test, -1)

    return x_train, x_val, x_test, y_train, y_val, y_test

```

علاوه بر MNIST، تصاویر فونتی ارقام ۱ تا ۹ نیز برای نزدیککردن داده آموزش به شکل ارقام چاپی جدول سودوکو استفاده شدند. تصاویر براساس پوشه کلاس بارگزاری، به تصویر خاکستری تبدیل و در اندازه 28×28 استاندارد شدند. پس زمینه و رقم تصاویر فونتی معکوس شد تا ساختار آن ها با خروجی مرحله پردازش تصویر یکسان باشد.

```

def get_data(data_choice, exclude=True):

    data_choice = data_choice.lower()
    if data_choice not in ['mnist', 'fonts', 'both']:
        raise ValueError("Invalid value for data_choice: {data_choice}. Valid options are: 'mnist', 'fonts', or 'both'")

    # Load MNIST data
    if data_choice == "mnist" or data_choice == "both":
        mnist_data = list(load_mnist_images())

    # Load font data
    if data_choice == "fonts" or data_choice == "both":
        if exclude:
            # Find the fonts to be excluded
            to_exclude = glob.glob("data/font_exclude/*.png")
            to_exclude = [fpath.split("\\")[-1] for fpath in to_exclude]
            to_exclude = [fpath.split("-")[-1] for fpath in to_exclude]
        else:
            to_exclude=None
        # Load the font images from file
        img_dict = get_font_image_dict(excluded_names=to_exclude)
        font_data = list(load_font_image_arrays(img_dict))

    # Assign x and y sets based on data_choice
    if data_choice == "mnist":
        x_train, x_val, x_test, y_train, y_val, y_test = mnist_data

    elif data_choice == "fonts":
        x_train, x_val, x_test, y_train, y_val, y_test = font_data

    elif data_choice == "both":
        # Combine MNIST and font digit images
        x_train = np.concatenate((font_data[0], mnist_data[0]))
        x_val = np.concatenate((font_data[1], mnist_data[1]))
        x_test = np.concatenate((font_data[2], mnist_data[2]))
        y_train = np.concatenate((font_data[3], mnist_data[3]))
        y_val = np.concatenate((font_data[4], mnist_data[4]))
        y_test = np.concatenate((font_data[5], mnist_data[5]))

    return x_train, x_val, x_test, y_train, y_val, y_test

```

برای افزایش تنوع داده‌ها، نمونه‌های دست نویس MNIST و تصاویر فونتی چاپی با یکدیگر ترکیب شدند. استفاده همزمان از این دو منبع باعث می‌شود مدل هم نسبت به تغییرات شکل دست نویس و هم نسبت به تفاوت فونت های چاپی مقاومتر باشد.

```
def _convert_digit_labels_to_10_classes(labels: np.ndarray) -> np.ndarray:
    labels = np.asarray(labels)

    if labels.ndim != 2:
        raise ValueError(f"Expected 2D one-hot labels, received {labels.shape}.")

    if labels.shape[1] == 9:
        digit_numbers = np.argmax(labels, axis=1) + 1
        return keras.utils.to_categorical(
            digit_numbers,
            num_classes=10,
        ).astype("float32")

    if labels.shape[1] == 10:
        return labels.astype("float32")

    raise ValueError(
        f"Expected 9- or 10-class labels, received {labels.shape[1]} classes."
    )
```

مدل نهایی به صورت یک طبقه بند ۱۰ کلاسه تعریف شد. کلاس صفر به خانه خالی و کلاس های ۱ تا ۹ به ارقام سودو کو اختصاص یافتند. داده های اولیه ارقام دارای ۹ کلاس بودند؛ بنابراین برچسب های آن ها به گونه ای بازنویسی شدند که اندیس خروجی هر کلاس با مقدار عدد متناظر یکسان باشد.


```

def _generate_synthetic_blank_images(count: int, seed: int) -> np.ndarray:
    if count <= 0:
        return np.empty((0, 28, 28, 1), dtype="float32")

    rng = np.random.default_rng(seed)
    images = np.zeros((count, 28, 28), dtype="float32")

    for index in range(count):
        image = rng.normal(loc=0.0, scale=0.012, size=(28, 28))
        image = np.clip(image, 0.0, 0.05)

        number_of_lines = int(rng.integers(0, 4))
        for _ in range(number_of_lines):
            intensity = float(rng.uniform(0.08, 0.40))
            thickness = int(rng.integers(1, 3))

            if rng.random() < 0.5:
                row = int(rng.choice([0, 1, 2, 25, 26, 27]))
                start = max(0, row - thickness + 1)
                end = min(28, row + thickness)
                image[start:end, :] = np.maximum(
                    image[start:end, :],
                    intensity,
                )
            else:
                column = int(rng.choice([0, 1, 2, 25, 26, 27]))
                start = max(0, column - thickness + 1)
                end = min(28, column + thickness)
                image[:, start:end] = np.maximum(
                    image[:, start:end],
                    intensity,
                )

        for _ in range(int(rng.integers(0, 5))):
            y = int(rng.integers(3, 25))
            x = int(rng.integers(3, 25))
            image[y, x] = float(rng.uniform(0.05, 0.25))

        images[index] = np.clip(image, 0.0, 1.0)

    return np.expand_dims(images, axis=-1)

```

ساخت تصاویر پایه، به تصاویر نویز ضعیف اضافه میشه. همچنین بقایای احتمالی خطوط جدول توی حاشیه شبیه‌سازی می‌شن. این کار باعث می‌شه خونه‌های خالی فقط شامل تصاویر کاملاً سیاه نباشه و مدل نسبت به نویز و قسمت‌های باقی‌مانده خطوط جدول مقاوم بشه.

```

def _load_real_blank_images() -> np.ndarray:
    patterns = ("*.png", "*.jpg", "*.jpeg", "*.bmp")
    paths: list[str] = []

    for pattern in patterns:
        paths.extend(glob.glob(str(REAL_BLANK_DIR / pattern)))

    loaded = []
    for path in sorted(paths):
        image = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
        if image is None:
            continue

        image = cv2.resize(image, (28, 28), interpolation=cv2.INTER_AREA)
        image = image.astype("float32") / 255.0

        if float(image.mean()) > 0.5:
            image = 1.0 - image

        loaded.append(np.expand_dims(np.clip(image, 0.0, 1.0), axis=-1))

    if not loaded:
        return np.empty((0, 28, 28, 1), dtype="float32")

    return np.asarray(loaded, dtype="float32")

def _blank_target_count(y_10: np.ndarray, ratio: float) -> int:
    labels = np.argmax(y_10, axis=1)
    digit_counts = [int(np.sum(labels == digit)) for digit in range(1, 10)]
    positive_counts = [count for count in digit_counts if count > 0]

    if not positive_counts:
        raise ValueError("No digit samples were found.")

    return max(1, int(round(float(np.median(positive_counts)) * ratio)))

```

برای تعریف کلاس خالی، از ترکیب نمونه های واقعی و مصنوعی استفاده شد. نمونه های واقعی از سلول های خالی تصاویر سودو کو استخراج شدند. برای افزایش تنوع، نمونه های مصنوعی نیز با افزودن نویزهای ضعیف، نقاط پراکنده و قسمت هایی از خطوط مرزی تولید شدند. این روش احتمال تشخیص اشتباه بقایای خطوط جدول به عنوان رقم را کاهش میدهد.

```
def _blank_target_count(y_10: np.ndarray, ratio: float) -> int:
    labels = np.argmax(y_10, axis=1)
    digit_counts = [int(np.sum(labels == digit)) for digit in range(1, 10)]
    positive_counts = [count for count in digit_counts if count > 0]

    if not positive_counts:
        raise ValueError("No digit samples were found.")

    return max(1, int(round(float(np.median(positive_counts)) * ratio)))
```

```
def _append_blank_class(
    x: np.ndarray,
    y: np.ndarray,
    blanks: np.ndarray,
    seed: int,
) -> tuple[np.ndarray, np.ndarray]:
    blank_labels = keras.utils.to_categorical(
        np.zeros(len(blanks), dtype=np.int64),
        num_classes=10,
    ).astype("float32")

    combined_x = np.concatenate((x, blanks), axis=0)
    combined_y = np.concatenate((y, blank_labels), axis=0)

    rng = np.random.default_rng(seed)
    order = rng.permutation(len(combined_x))
    return combined_x[order], combined_y[order]
```

تعداد هدف برای کلاس خانه خالی براساس میانه تعداد نمونه های کلاس های رقمی تعیین می شود. برچسب کلاس خالی ساخته میشه. سپس نمونه های خانه خالی به داده های رقم اضافه می شن. و در پایان داده ها به صورت تصادفی بازبایی می شن.

```

251 def get_data(
252     data_choice: str,
253     exclude: bool = True,
254     blank_ratio: float = 1.0,
255     random_state: int = 2026,
256 ):
257     if blank_ratio <= 0:
258         raise ValueError("blank_ratio must be greater than zero.")
259
260     x_train, x_val, x_test, y_train, y_val, y_test = _load_base_data(
261         data_choice=data_choice,
262         exclude=exclude,
263     )
264
265     x_train = _ensure_image_shape_and_scale(x_train)
266     x_val = _ensure_image_shape_and_scale(x_val)
267     x_test = _ensure_image_shape_and_scale(x_test)
268
269     y_train = _convert_digit_labels_to_10_classes(y_train)
270     y_val = _convert_digit_labels_to_10_classes(y_val)
271     y_test = _convert_digit_labels_to_10_classes(y_test)
272
273     targets = (
274         _blank_target_count(y_train, blank_ratio),
275         _blank_target_count(y_val, blank_ratio),
276         _blank_target_count(y_test, blank_ratio),
277     )
278
279     real_blanks = _load_real_blank_images()
280     real_train, real_val, real_test = _split_real_blanks(
281         real_blanks,
282         target_counts=targets,
283         seed=random_state,
284     )
285
286     blank_train = _make_blank_split(real_train, targets[0], random_state + 1)
287     blank_val = _make_blank_split(real_val, targets[1], random_state + 2)
288     blank_test = _make_blank_split(real_test, targets[2], random_state + 3)
289
290     x_train, y_train = _append_blank_class(
291         x_train, y_train, blank_train, random_state + 11
292     )
293     x_val, y_val = _append_blank_class(
294         x_val, y_val, blank_val, random_state + 12
295     )

```

ییس از دریافت داده های ارقام، ابعاد و دامنه مقادیر تصاویر کنترل شد. برچسبها به ساختار دهکلاسه تبدیل شدند و نمونه های خانه خالی به مجموعه های آموزش، اعتبارسنجی و آزمون اضافه شد. در پایان، داده ها به صورت تصادفی مرتب شدند تا نمونه های هر کلاس در سراسر مجموعه توزیع بشن.

```
def build_ten_class_model() -> keras.Model:
    augmentation = keras.Sequential(
        [
            layers.RandomRotation(0.03, fill_mode="constant", fill_value=0.0),
            layers.RandomTranslation(0.06, 0.06, fill_mode="constant", fill_value=0.0),
            layers.RandomZoom((-0.06, 0.06), (-0.06, 0.06), fill_mode="constant", fill_value=0.0),
        ],
        name="data_augmentation",
    )
```

به منظور افزایش تنوع داده آموزشی و کاهش بیش برآزش، از روش افزایش داده استفاده شد. تصاویر در حین آموزش تحت چرخش، انتقال و تغییر مقیاس محدود قرار گرفتند. دامنه این تغییرات کوچک انتخاب شد تا شکل اصلی رقم حفظ شود. از بازتاب افقی یا عمودی استفاده نشد، زیرا این تبدیل‌ها می‌توانند هندسه ارقام را غیرواقعی کنند.

```
return keras.Sequential(
    [
        keras.Input(shape=(28, 28, 1), name="cell_image"),
        augmentation,
        layers.Conv2D(32, (3, 3), activation="relu", name="conv_1"),
        layers.MaxPooling2D((2, 2), name="pool_1"),
        layers.Conv2D(64, (3, 3), activation="relu", name="conv_2"),
        layers.MaxPooling2D((2, 2), name="pool_2"),
        layers.Flatten(name="flatten"),
        layers.Dropout(0.5, name="dropout"),
        layers.Dense(10, activation="softmax", name="class_output"),
    ],
    name="sudoku_10class_transfer_model",
)
```

مدل تشخیص سلول با استفاده از یک شبکه عصبی کانولوشنی سبک طراحی شد. شبکه شامل دو لایه کانولوشنی با ۳۲ و ۶۴ فیلتر، دو لایه Max Pooling، یک لایه Flatten، یک لایه Dropout و یک لایه خروجی Softmax است. لایه dropout با نرخ ۰.۵ برای کاهش بیش برآزش به مدل اضافه شد. لایه خروجی دارای ۱۰ نورون است که کلاس صفر را به خانه خالی و کلاس‌های ۱ تا ۹ را به ارقام اختصاص می‌دهد.

```
def transfer_weights(old_model: keras.Model, new_model: keras.Model) -> None:
    old_convs, old_dense = weighted_layers(old_model)
    new_convs, new_dense = weighted_layers(new_model)

    for old_layer, new_layer in zip(old_convs, new_convs):
        if [w.shape for w in old_layer.get_weights()] != [w.shape for w in new_layer.get_weights()]:
            raise ValueError(f"Convolution shape mismatch: {old_layer.name} -> {new_layer.name}")
        new_layer.set_weights(old_layer.get_weights())

    old_kernel, old_bias = old_dense.get_weights()
    new_kernel, new_bias = new_dense.get_weights()

    if old_kernel.shape[1] != 9 or new_kernel.shape[1] != 10:
        raise ValueError(
            f"Expected old Dense=9 and new Dense=10, received {old_kernel.shape} and {new_kernel.shape}."
        )
    if old_kernel.shape[0] != new_kernel.shape[0]:
        raise ValueError("Dense feature dimensions do not match.")

    new_kernel[:, 0] = 0.0
    new_bias[0] = 0.0
    new_kernel[:, 1:] = old_kernel
    new_bias[1:] = old_bias
    new_dense.set_weights([new_kernel, new_bias])
```

برای استفاده از دانش مدل قبلی، وزن های لایه های استخراج ویژگی و وزن های مربوط به ارقام ۱ تا ۹ از مدل ۹ کلاسه به مدل جدید منتقل شدند. در مدل جدید، ستون صفر لایه خروجی به کلاس خالی اختصاص یافت و به صورت اولیه صفر گذاری شد. بدین ترتیب، مدل ده کلاسه بدون از دست دادن توانایی قبلی در تشخیص ارقام ساخته شد.

```
def compile_model(model: keras.Model, learning_rate: float) -> None:
    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=learning_rate),
        loss="categorical_crossentropy",
        metrics=["accuracy"],
    )
```

مدل با استفاده از بهینه ساز Adam آموزش داده شد. تابع هزینه از نوع Categorical Cross-Entropy انتخاب گردید که برای مسائل طبقه بندی چند کلاسه با خروجی Softmax مناسب است. معیار Accuracy نیز برای محاسبه نسبت نمونه های درست طبقه بندی شده مورد استفاده قرار گرفت.

```

116
117     checkpoint = keras.callbacks.ModelCheckpoint(
118         filepath=str(out_path),
119         monitor="val_accuracy",
120         mode="max",
121         save_best_only=True,
122         verbose=1,
123     )
124
125     print("\nStage 1: train the new blank/output head...")
126     freeze_features(model, True)
127     compile_model(model, 1e-3)
128     history_head = model.fit(
129         x_train,
130         y_train,
131         validation_data=(x_val, y_val),
132         epochs=args["head_epochs"],
133         batch_size=args["batch_size"],
134         callbacks=[checkpoint],
135         shuffle=True,
136         verbose=1,
137     )

```

بهترین مدل بر اساس دقت val ذخیره میشه. سپس لایه های استخراج ویژگی freeze میشه. مدل با نرخ یادگیری ۰.۰۰۱ کامپایل می شه و آموزش مرحله اول شروع می شه. در این مرحله، بیشتر لایه های قبلی ثابت هستند و لایه خروجی ده کلاس آموزش می بیند.

```

138
139     print("\nStage 2: fine-tune the complete ten-class model...")
140     freeze_features(model, False)
141     compile_model(model, 1e-4)
142     callbacks = [
143         checkpoint,
144         keras.callbacks.EarlyStopping(
145             monitor="val_loss", patience=4, restore_best_weights=True, verbose=1
146         ),
147         keras.callbacks.ReduceLROnPlateau(
148             monitor="val_loss", factor=0.5, patience=2, min_lr=1e-6, verbose=1
149         ),
150     ]
151     history_fine = model.fit(
152         x_train,
153         y_train,
154         validation_data=(x_val, y_val),
155         epochs=args["fine_tune_epochs"],
156         batch_size=args["batch_size"],
157         callbacks=callbacks,
158         shuffle=True,
159         verbose=1,
160     )
161

```

آموزش مدل در دو مرحله انجام شد. در مرحله نخست، لایه های کانولوشنی ثابت نکه داشته شدند و تنها بخش جدید خروجی آموزش داده شد. در مرحله دوم، تمام لایهها آزاد شدند و شبکه با نرخ یادگیری کوچکتر -Fine tune کردید. از early stopping برای جلوگیری از بیش برآزش و از ReduceLROnPlateau برای کاهش نرخ یادگیری در صورت توقف بهبود استفاده شد.

پس از پایان آموزش، بهترین مدل براساس دقت اعتبارسنجی بارگزاری و روی مجموعه آزمون مستقل ارزیابی شد. مقادیر Test Loss و Test Accuracy همراه با مشخصات مدل و تعداد دورههای آموزش در یک فایل JSON ذخیره شدند.

در مرحله استنتاج، هر ۸۱ سلول استخراج شده به ابعاد $1 \times 28 \times 28$ تبدیل و به صورت یک Batch به مدل ارائه شدند. مدل برای هر سلول برداری شامل ۱۰ احتمال تولید می‌کند. کلاسی که بیشترین احتمال را دارد به‌عنوان خروجی انتخاب می‌شود. کلاس صفر نشان دهنده خانه خالی و کلاس های ۱ تا ۹ نشان دهنده ارقام هستند. در پایان، خروجی ۸۱ سلول به ماتریس 9×9 تبدیل می‌شود.


```

645 def get_predicted_sudoku_grid(model, cells):
653
654     if len(cells) != 81:
655         raise ValueError(
656             f"Expected 81 Sudoku cells, received {len(cells)}."
657         )
658
659     cell_images = np.array(
660         [np.expand_dims(cell["img"], axis=-1) for cell in cells],
661         dtype="float32",
662     )
663
664     while cell_images.ndim > 4 and cell_images.shape[-1] == 1:
665         cell_images = np.squeeze(cell_images, axis=-1)
666
667     if cell_images.shape != (81, 28, 28, 1):
668         raise ValueError(
669             "Expected input shape (81, 28, 28, 1), "
670             f"received {cell_images.shape}."
671         )
672
673     if cell_images.size and float(cell_images.max()) > 1.0:
674         cell_images /= 255.0
675
676     probabilities = model.predict(cell_images, verbose=0)
677
678     if probabilities.ndim != 2 or probabilities.shape[1] != 10:
679         raise ValueError(
680             "A ten-class model is required. Expected output shape "
681             f"(N, 10), received {probabilities.shape}."
682         )
683
684     predicted_labels = np.argmax(probabilities, axis=1).astype(int)
685
686     for index, (cell, predicted_label) in enumerate(
687         zip(cells, predicted_labels)
688     ):
689         cell["contains_digit"] = bool(predicted_label != 0)
690         cell["predicted_class"] = int(predicted_label)
691         cell["prediction_confidence"] = float(
692             probabilities[index, predicted_label]
693         )
694
695     return predicted_labels.reshape((9, 9))

```

پس از استخراج و مرتب سازی سلولها، تصاویر هر ۸۱ خانه در قالب یک آرایه چهاربعدی با ابعاد $۸۱ \times ۲۸ \times ۲۸ \times ۱$ تشکیل شدند. مدل CNN برای هر سلول یک بردار احتمال دهعضوی تولید کرد. اندیس بیشترین احتمال به عنوان کلاس نهایی انتخاب شد. کلاس صفر نشان دهنده خانه خالی و کلاس های ۱ تا ۹ متناظر با ارقام سودوکو هستند. در پایان، خروجی ۸۱ تایی مدل به یک ماتریس ۹×۹ تبدیل و برای حل به الگوریتم Backtracking ارسال شد. در نمایش عددی جدول، مقدار صفر برای خانه های خالی در نظر گرفته شد. این قرارداد موجب می شود الگوریتم حل کننده بتواند خانه های تکمیل نشده را به سادگی پیدا کند. ارقام اولیه جدول با مقادیر ۱ تا ۹ و خانه های خالی با مقدار صفر در ماتریس ذخیره شدند.

خروجی مرحله تشخیص رقم در متغیر `grid_array` ذخیره شد. برای امکان بررسی خطاهای شناسایی، این ماتریس.بیش از حل در فایل `predicted_grid.txt_۰۹` ذخیره می شود. سپس یک نسخه کپی از ماتریس به کلاس حل کننده ارسال می گردد تا جدول اصلی شناسایی شده برای تحلیل های بعدی بدون تغییر باقی بماند.

ماتریس جدول به یک نمونه از کلاس `Sudokusolver` داده شد. متد `solve()` الگوریتم بازگشتی حل را اجرا می کند و بسته به وجود یا عدم وجود پاسخ معتبر، مقدار منطقی درست یا نادرست برمیگرداند.

```

2
3
4 def is_valid_number(self, board, number, position):
5     """
6     Check whether or not it is valid to insert a particular number at a
7     specified position on the board.
8
9     Args:
10        board (list[list[int]]) - list of lists representing current puzzle state
11        number (int) - the number we are testing for validity
12        position (tuple) - the position on the board where number is being inserted
13
14    Returns:
15        valid (bool) - whether or not the number is valid in the stated position
16    """
17
18    # Assign the board dimensions to variables. Assume a square board.
19    num_rows = len(board)
20    square_size = int(math.sqrt(num_rows))
21
22    # Unpack row and column from position tuple for readability
23    row_idx, col_idx = position
24
25    # Check if number is already present in current row
26    if number in board[row_idx]:
27        return False
28
29    # Check if number is already present in current column
30    current_column_values = [board[row][col_idx] for row in range(num_rows)]
31    if number in current_column_values:
32        return False
33
34    # Get indices of the square that our position lies in
35    square_x_idx = col_idx // square_size
36    square_y_idx = row_idx // square_size
37    # Check if number is already present in current square
38    for row in range(square_y_idx * square_size, (square_y_idx * square_size) + square_size):
39        for col in range(square_x_idx * square_size, (square_x_idx * square_size) + square_size):
40            if board[row][col] == number and (row, col) != position:
41                return False
42
43    # If we reach this point, the number is valid
44    return True

```

پیش از قرار دادن هر عدد در خانه خالی، سطر متناظر بررسی می شود. اگر عدد پیشنهادی از قبل در سطر وجود داشته باشد، آن انتخاب رد شده و الگوریتم عدد بعدی را امتحان میکند.

در مرحله بعد، تمام مقادیر ستون متناظر باخانه موردنظر استخراج می شوند. اگر عدد پیشنهادی در ستون وجود داشته باشد، قرار دادن آن با قوانین سودوکو مغایرت داشته و رد می شود.

برای بررسی محدودیت بلوک، شماره بلوک افقی وعمودی با استفاده از تقسیم صحیح شماره سطر وستون بر اندازه

بلوک محاسبه شد. سپس تمام خانه های بلوک 3×3 بیمایش شدند. در صورت وجود عدد پیشنهادی در بلوک، انتخاب رد میشود. تنها زمانی مقدار درست بازگردانده می شود که عدد در سطر، ستون و بلوک تکراری نباشد.

الگوریتم حل بهصورت بازگشتی پیادهسازی شد. ابتدا اولین خانه خالی انتخاب می شود و اعداد ۱ تا ۹ به ترتیب در آن امتحان می شوند. برای هر عدد، محدودیت های سطر، ستون و بلوک بررسی می شوند. اگر عدد معتبر باشد، در خانه قرار گرفته و حل برای خانه بعدی ادامه پیدا می کند. اگر در مراحل بعدی هیچ انتخاب معتبری وجود نداشته باشد، الگوریتم به مرحله قبلی بازمی گردد، مقدار خانه را صفر میکند و عدد بعدی را امتحان می نماید. این فرآیند تا یافتن پاسخ یا اثبات نبود پاسخ ادامه پیدا می کند.

```
if solved and not np.any(np.asarray(solver.board) == 0):  
    diagnostics["status"] = "success"  
    diagnostics["reason"] = "Grid extraction, digit recognition and solving succeeded."  
    print("Success - sudoku solved!")
```

پس از پایان الگوریتم، مقدار بازگشتی متد حل و وجود صفر در جدول بررسی شدند. تنها در صورتی نتیجه موفق اعلام می شود که الگوریتم مقدار درست برگرداند و هیچ خانه خالی در جدول باقی نمانده باشد.

اگر الگوریتم نتواند همه خانه ها را تکمیل کند، وضعیت اجرا به عنوان solver_failed ثبت می شود. این حالت ممکن است ناشی از نبود پاسخ برای جدول یا تشخیص اشتباه یک یا چند رقم توسط مدل باشد. علت شکست در فایل گزارش ذخیره شده و جدول حاصل نیز برای تحلیل خطا چاپ می شود.

پس از حل جدول، تنها مقادیر مربوط به خانه های اولیه خالی روی یک تصویر سفید متناظر با جدول Perspective شده نوشته شدند. سپس با استفاده از ماتریس تبدیل پرسکتیو و گزینه تبدیل معکوس، ارقام حل شده به مختصات تصویر اصلی بازگردانده شدند. پاسخ ها با رنگ متمایز روی تصویر ورودی نمایش داده و در فایل solution.png_۱۰ ذخیره شدند.